Name: Kamran Gul

CMS: 023-25-0161

Section: C

OOP Lab: 10

Task 1

Banking —BankAccount, SavingsAccount, CurrentAccount

Real-life story:

A bank offers **accounts**. Every account has an **account number** and **holder name**. The bank applies **different rules** for **savings** (interest on deposits) and **current** (checking-style: often **no** or **very low** interest, but **higher daily movement**). All are still **bank accounts** for the software.

Classes (three):

Class	Role
`BankAccount`	General account: default interest and default **daily withdrawal limit**.
`SavingsAccount`	**Savings** product: **higher** interest; withdrawal limit may be **stricter** (bank policy).
`CurrentAccount`	**Current / checking** product: **lower or zero** interest; **higher** withdrawal limit for frequent transactions.

Fields (private, suggested):

Class	Fields
`BankAccount`	`accountNumber` (String), `holderName` (String).
`SavingsAccount`	No extra fields required, or optional `minimumBalance` if you want (not required).
`CurrentAccount`	No extra fields required.

Superclass — `BankAccount` — two methods:

Method	Real-life meaning	What the code should do
`display()`	Mini statement: who owns this account.	Print **account number** and **holder name** (and optionally a label like `"Account type: General"`).

Method	Real-life meaning	What the code should do
`annualInterestRate()`	Default yearly interest **percent** for a generic account.	Return a `double`, e.g. **0.5** (meaning 0.5% per year — document your convention in a comment).

Subclass — `SavingsAccount extends BankAccount`:

Method	Real-life meaning	What the code should do
(constructor)	Open a savings account.	First line: `super(accountNumber, holderName);`
`annualInterestRate()` **(override)**	Savings should earn **more** interest than the default.	Return a **higher** rate than `BankAccount` (e.g. **4.0**). Use `@Override`.

Subclass — `CurrentAccount extends BankAccount`:

Method	Real-life meaning	What the code should do
(constructor)	Open a current account.	First line: `super(accountNumber, holderName);`
`annualInterestRate()` **(override)**	Current accounts often pay **no** or **minimal** interest.	Return **0.0** or **0.1** — **lower** than savings and **not higher** than the parent's default if your story is "no interest". Use `@Override`.

Optional (only if your instructor allows extra methods): override `display()` in one subclass to add `"Savings"` / `"Current"` using `super.display()` first.

In `Main`:

- Create: `new BankAccount(...)`, `new SavingsAccount(...)`, `new CurrentAccount(...)`.
- For each, call `display()` and print `annualInterestRate()`.
- Show that **savings** interest > generic ≥ current (adjust numbers so this is true).

Solution

```java
class BankAccount {
    private String accountNumber;
    private String holderName;

    public BankAccount(String accountNumber, String holderName) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
    }

    public void display() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Holder Name: " + holderName);
    }

    public double annualInterestRate() {
        return 0.5; // 0.5%
```

```java
    }
}

class SavingsAccount extends BankAccount {

    public SavingsAccount(String accountNumber, String holderName) {
        super(accountNumber, holderName);
    }

    @Override
    public double annualInterestRate() {
        return 4.0; // higher interest
    }
}

class CurrentAccount extends BankAccount {

    public CurrentAccount(String accountNumber, String holderName) {
        super(accountNumber, holderName);
    }

    @Override
    public double annualInterestRate() {
        return 0.0; // no interest
    }
}
public class Main {
    public static void main(String[] args) {
        BankAccount b = new BankAccount("001", "Ali");
        SavingsAccount s = new SavingsAccount("002", "Ahmed");
        CurrentAccount c = new CurrentAccount("003", "Sara");

        System.out.println("=== Bank Account ===");
        b.display();
        System.out.println("Interest: " + b.annualInterestRate());

        System.out.println("\n=== Savings Account ===");
        s.display();
        System.out.println("Interest: " + s.annualInterestRate());

        System.out.println("\n=== Current Account ===");
        c.display();
        System.out.println("Interest: " + c.annualInterestRate());
    }
}
```

Output

```
=== Bank Account ===
Account Number: 001
Holder Name: Ali
Interest: 0.5

=== Savings Account ===
Account Number: 002
Holder Name: Ahmed
Interest: 4.0

=== Current Account ===
Account Number: 003
Holder Name: Sara
Interest: 0.0
```

Task 2

Cafeteria — Meal, StudentMeal, FacultyMeal

Real-life story:

The cafeteria sells **meals** with a **menu name** and a **list price**. **Student meals** and **faculty meals** are still **meals**, but the **price charged** differs: students get a **large discount**, faculty often get a **moderate** discount compared with the public list price.

Classes (three):

Class	Role
`Meal`	**List** meal: name + **standard** price (walk-in / guest).
`StudentMeal`	Same meal concept; **student** pricing.
`FacultyMeal`	Same meal concept; **faculty** pricing.

Fields (private, suggested):

Class	Fields
`Meal`	`mealName` (String), `listPrice` (double).
`StudentMeal`	Use `super(mealName, listPrice)` — price rules in `price()` override.
`FacultyMeal`	Same.

Superclass — `Meal` — two methods:

Method	Real-life meaning	What the code should do
`display()`	Show meal name and list price.	Print **meal name** and **list price**.
`price()`	Full price (guest / no discount).	Return **`listPrice`** unchanged (the "sticker" price).

Subclass — `StudentMeal extends Meal`:

Method	Real-life meaning	What the code should do
(constructor)	e.g. `StudentMeal(String mealName, double listPrice)`	First line: `super(mealName, listPrice);`
`price()` **(override)**	Student pays **less** — e.g. **50%** of list.	Return `listPrice * 0.50` (or use `super.price()` and multiply). Document the rule in a comment.

Subclass — `FacultyMeal extends Meal`:

Method	Real-life meaning	What the code should do
(constructor)	e.g. `FacultyMeal(String mealName, double listPrice)`	First line: `super(mealName, listPrice);`
`price()` **(override)**	Faculty pays **less than list** but **usually more than student**.	Return e.g. **`listPrice * 0.75`** — must be > student price and < list for the same `listPrice`.

In `Main`:

- Use the **same** `mealName` and `listPrice` for all three (e.g. `"Biryani"`, `300`).
- Create `Meal`, `StudentMeal`, `FacultyMeal`.
- Print `display()` for each and print `price()` for each — show **student < faculty < guest list** (or **student ≤ faculty < list**).

Solution

```
class Meal {
    private String mealName;
    private double listPrice;

    public Meal(String mealName, double listPrice) {
        this.mealName = mealName;
        this.listPrice = listPrice;
    }

    public void display() {
        System.out.println("Meal: " + mealName);
        System.out.println("List Price: " + listPrice);
    }

    public double price() {
        return listPrice;
    }
}

class StudentMeal extends Meal {

    public StudentMeal(String mealName, double listPrice) {
        super(mealName, listPrice);
    }

    @Override
    public double price() {
        return super.price() * 0.50; // 50% discount
    }
}

class FacultyMeal extends Meal {

    public FacultyMeal(String mealName, double listPrice) {
        super(mealName, listPrice);
    }

    @Override
```

```java
    public double price() {
        return super.price() * 0.75; // 25% discount
    }
}

public class Main {
    public static void main(String[] args) {
        Meal m = new Meal("Biryani", 300);
        StudentMeal sm = new StudentMeal("Biryani", 300);
        FacultyMeal fm = new FacultyMeal("Biryani", 300);

        System.out.println("=== Meal ===");
        m.display();
        System.out.println("Price: " + m.price());

        System.out.println("\n=== Student Meal ===");
        sm.display();
        System.out.println("Price: " + sm.price());

        System.out.println("\n=== Faculty Meal ===");
        fm.display();
        System.out.println("Price: " + fm.price());
    }
}
```

Output

```
=== Meal ===
Meal: Biryani
List Price: 300.0
Price: 300.0

=== Student Meal ===
Meal: Biryani
List Price: 300.0
Price: 150.0

=== Faculty Meal ===
Meal: Biryani
List Price: 300.0
Price: 225.0
```

Task 3

Hotel — HotelRoom, StandardRoom, Suite

Real-life story:

A hotel manages **rooms**. Every booking has a **room code** (e.g. `"204"`). **Standard rooms** and **suites** are both **hotel rooms**, but **nightly rent** differs: standard is **mid** range; **suite** is **premium**. You can also model a **base** `HotelRoom` as the cheapest "economy" or generic tier if you prefer — the tables below use **base = lowest** rent.

Classes (three):

Class	Role
`HotelRoom`	**Base** room tier: smallest rent (economy / default).
`StandardRoom`	**Standard** tier: **higher** than base.
`Suite`	**Suite** tier: **highest** rent.

Fields (private, suggested):

Class	Fields
`HotelRoom`	`roomCode` (String).
`StandardRoom`	`super(roomCode)` only, or add `bedCount` (int) optionally.
`Suite`	`super(roomCode)` only, or add `livingRooms` (int) optionally.

Superclass — `HotelRoom` — two methods:

Method	Real-life meaning	What the code should do
`display()`	Show which room this is.	Print **room code** only (e.g. `"Room: 101"`). Do **not** put `"Standard"` or `"Suite"` here unless you also **override** `display()` in those subclasses — otherwise one shared `display()` is fine for all three types.
`nightlyRate()`	Nightly rent for this **tier** (base = cheapest in this lab).	Return a `double`, e.g. **3000.0** for the base tier.

Subclass — `StandardRoom extends HotelRoom`:

Method	Real-life meaning	What the code should do
(constructor)	e.g. `StandardRoom(String roomCode)`	First line: `super(roomCode);`

Method	Real-life meaning	What the code should do
`nightlyRate()` **(override)**	Standard room costs **more** than base.	Return e.g. **5500.0** — > base rate.
Optionally override `display()`	Show `"Standard"` in addition to room code.	Optional: call `super.display()` then print one line.

Subclass — `Suite extends HotelRoom`:

Method	Real-life meaning	What the code should do
(constructor)	e.g. `Suite(String roomCode)`	First line: `super(roomCode);`
`nightlyRate()` **(override)**	Suite is **most expensive**.	Return e.g. **12000.0** — > standard > base.
Optionally override `display()`	Show `"Suite"`.	Optional: call `super.display()` then print one line.

In `Main`:

- Create one `HotelRoom`, one `StandardRoom`, one `Suite` (you may use different room codes, e.g. `"101"`, `"205"`, `"501"`).
- For each, call `display()` and print `nightlyRate()`.
- Show **base < standard < suite** rates.

Solution

```java
class HotelRoom {
    private String roomCode;

    public HotelRoom(String roomCode) {
        this.roomCode = roomCode;
    }

    public void display() {
        System.out.println("Room Code: " + roomCode);
    }

    public double nightlyRate() {
        return 3000.0; // base rate
    }
}

class StandardRoom extends HotelRoom {

    public StandardRoom(String roomCode) {
        super(roomCode);
    }

    @Override
    public double nightlyRate() {
        return 5500.0;
```

```java
    }
}

class Suite extends HotelRoom {

    public Suite(String roomCode) {
        super(roomCode);
    }

    @Override
    public double nightlyRate() {
        return 12000.0;
    }
}

public class Main {
    public static void main(String[] args) {
        HotelRoom h = new HotelRoom("101");
        StandardRoom s = new StandardRoom("205");
        Suite su = new Suite("501");

        System.out.println("=== Hotel Room ===");
        h.display();
        System.out.println("Rate: " + h.nightlyRate());

        System.out.println("\n=== Standard Room ===");
        s.display();
        System.out.println("Rate: " + s.nightlyRate());

        System.out.println("\n=== Suite ===");
        su.display();
        System.out.println("Rate: " + su.nightlyRate());
    }
}
```

Output

```
=== Hotel Room ===
Room Code: 101
Rate: 3000.0

=== Standard Room ===
Room Code: 205
Rate: 5500.0

=== Suite ===
Room Code: 501
Rate: 12000.0
```